PyQuest: Product Requirements and Technical Requirements

Working title "PyQuest" (placeholder). Status: draft for review, revision R1 applied (Part 3 lists every change; where Part 3 and earlier text differ, Part 3 prevails). Planning only; no implementation has started.

Part 1: Product Requirements Document (PRD)

1. Executive Summary

PyQuest is a gamified, browser-based platform for learning Python. It combines structured lessons, an in-browser code editor with real execution, auto-graded challenges, and game mechanics (XP, levels, streaks, badges) with an AI tutor that nudges learners to reason instead of handing over answers.

Vision: make a first Python course feel like playing a well-designed game while still teaching real, transferable skills.

Core bet: beginners stay motivated when the loop *read → write code → run → get instant feedback → earn progress* is fast, forgiving, and visibly rewarding.

Operating constraint: the MVP should run at or near ₹0, so all code execution happens in the learner's browser (Pyodide) and the MVP tutor uses authored hints and rule-based error explanations only (no LLM).

2. Assumptions (to confirm)

#	Assumption	Impact if wrong
A1	Python is the only language in v1.	Multi-language needs server-side sandboxing.
A2	English-only UI and content in v1.	Needs i18n framework early.
A3	Greenfield repository (no existing code). Phase 0 will verify this.	Plan must adapt to existing conventions.
A4	One developer plus one AI coding agent.	Roadmap pacing changes.

#	Assumption	Impact if wrong
A5	Audience is mostly India-based students on mid-range phones and variable bandwidth.	Performance budgets matter more.
A6	Learners may be minors (school students).	Privacy and consent obligations rise (see TRD security plan).
A7	Free-tier quotas and prices change; every figure marked *verify* must be checked against official docs before commitment.	Budget surprises.
A8	No payments, no teacher/classroom features in v1.	Scope growth.

3. Target Users and Personas

1. **Aarav, 15, school student.** First programming exposure, phone or shared laptop, short sessions. Needs tiny steps, friendly errors, visible wins.
2. **Meera, 19, college student.** Taking a Python course, needs practice before lab exams. Wants challenges with tests and timed rounds.
3. **Rohan, 21, placement/assessment prep.** Comfortable with basics, wants graded problems, difficulty ladder, daily challenge.
4. **Content author (internal).** Adds lessons and challenges by editing validated files. Needs fast feedback on schema errors.

4. Goals, Non-Goals, Success Metrics

Goals

- G1: A beginner completes their first coding exercise within 3 minutes of landing.
- G2: Learning content is data-driven and extensible without touching UI code.
- G3: Progress, XP, and achievements are recorded server-side and idempotently, and are honestly labeled self-reported until trusted grading exists.
- G4: The product feels professionally designed and accessible.
- G5: Infrastructure cost for MVP stays within free tiers, with usage caps instead of surprise bills.

Non-goals (v1)

- Server-side code execution, languages other than Python, mobile native apps, payments, classroom/teacher tools, social features beyond an optional leaderboard, a full admin dashboard.

Success metrics (targets are planning assumptions, to be tuned after launch)

Metric	Initial target
Landing → first code run	≥ 35% of visitors who click Start
First lesson completion	≥ 60% of registered users
Day-7 retention	≥ 20%
Run-code success (no platform error)	≥ 99%
Time to first Pyodide-ready editor (cached)	≤ 4 s p75, warm cache, R1 mid-range profile (R1-D)
Lighthouse accessibility score	≥ 95 on key pages
Duplicate XP awards	0

5. MVP Scope and Deferred Features

Smallest valuable MVP ("Launch Candidate 1")

A learner can: land on the site → sign up → follow one beginner course (about 3 modules and 12 lessons (assumption)) → edit and run Python in-browser → pass auto-graded challenges → earn XP once, level up, keep a streak, and see progress after signing in again. The tutor is the authored Help panel (R1-C).

Area	MVP (v1)	Deferred
Landing page	Yes	A/B tests, interactive demo variants
Dashboard	XP, level, streak, continue card, progress	Recommendations beyond "next lesson"
Courses/lessons	1 course, JSON/MDX content in repo	Multiple courses, community content

Area	MVP (v1)	Deferred
Editor + Pyodide	Yes (worker, timeout, cancel)	`input()` interactivity polish, package installs, plotting
Challenges	Easy/Medium, bundled tests run in the browser (practice check); no secret tests	Trusted server-side grading, timed rounds, multi-file problems
Gamification	XP, levels, streaks, bronze/silver/gold badges, achievement toasts	Leaderboards (need trusted grading), debugging dungeons, daily challenge, streak freezes, module unlocks
AI tutor	Help panel: authored hints, Python error explanations, per-test feedback (no free-text chat, no LLM)	Optional LLM tutor (post-launch, feature flag, quotas, adults only until privacy review)
Auth	Email/password only, with email verification and an age gate	Social login, parental-consent flow (pending decision D1)
Admin	Repo-based content with CI schema validation	Admin dashboard (revisit after ~50 lessons)

Decision (admin): a dashboard is not needed in v1. Content as validated files in Git gives review, history, and rollback for free.

6. Functional Requirements

Priority: **P0** = MVP-blocking, **P1** = should ship in v1.x, **P2** = later.

A. Landing page

ID	Requirement	Pri
FR-A1	Hero with value proposition and primary CTA ("Start free"; "Continue" when signed in).	P0
FR-A2	Embedded mini editor that runs a short Python snippet without signup.	P1
FR-A3	Course preview cards, feature highlights, footer with legal links.	P0
FR-A4	Fully responsive; no layout shift on load.	P0

B. Student dashboard

ID	Requirement	Pri
FR-B1	Welcome header, XP, level with progress-to-next-level bar, current streak.	P0
FR-B2	Continue-learning card linking to the exact next incomplete lesson.	P0
FR-B3	Course progress (completed/total lessons), recent achievements.	P0
FR-B4	Recommended next activity (rule: next lesson, or an unattempted challenge matching level).	P1

C. Courses and lessons

ID	Requirement	Pri
FR-C1	Hierarchy: Course → Module → Lesson, rendered from validated content files.	P0
FR-C2	Lesson page: objectives, explanation, runnable examples, guided steps, exercise, hints, solution (revealed after attempts or on request).	P0
FR-C3	Completion tracking, previous/next navigation, locked/unlocked states.	P0

ID	Requirement	Pri
FR-C4	Content schema validated at build time; invalid content fails CI.	P0

D. Coding environment

ID	Requirement	Pri
FR-D1	Monaco editor: Python highlighting, line numbers, auto-indent, starter code, Reset.	P0
FR-D2	Run executes in Pyodide inside a Web Worker; stdout, stderr, and traceback shown in separate panels.	P0
FR-D3	Execution timeout (default 5 s), user-initiated Stop that terminates the worker, automatic worker restart.	P0
FR-D4	States: loading runtime, running, timed out, cancelled, error.	P0
FR-D5	Friendly error layer: maps common Python exceptions to plain-language explanations.	P1
FR-D6	Clear note on limits (supported stdlib, no network, no arbitrary packages).	P0
FR-D7	Editor usable on tablet; mobile uses a stacked, simplified layout.	P0

E. Challenges

ID	Requirement	Pri
FR-E1	Challenge page: description, I/O examples, starter code, difficulty, XP value.	P0
FR-E2	Visible sample tests run on "Run tests"; per-test pass/fail with diffs.	P0

ID	Requirement	Pri
FR-E3	Additional edge-case checks run with the sample tests on Submit. They ship with the content, so they are not secret; results are labeled a practice check.	P0
FR-E4	Progressive hints (max 3 levels), free in the MVP (no XP cost).	P1
FR-E5	Submission history, best result, completion state.	P0
FR-E6	Timed rounds and daily challenge.	P2 (v1.1)

F. Gamification

ID	Requirement	Pri
FR-F1	XP is written only by the server to an append-only ledger, once per (user, source) pair, with amounts from server config and a trust level of self_reported (MVP) or trusted (future).	P0
FR-F2	Level curve defined in config; level derived from total XP, never stored independently.	P0
FR-F3	Daily streak based on the learner's timezone; no streak freezes in the MVP (deferred).	P0
FR-F4	Badges with bronze/silver/gold tiers and published unlock rules.	P0
FR-F5	Achievement toast and badge gallery.	P0
FR-F6	Unlockable areas: next module unlocks on prior module completion.	P1
FR-F7	Debugging dungeons (fix broken code); leaderboard (not launched until trusted grading exists).	P2

G. AI tutor

ID	Requirement	Pri
FR-G1	Help panel on lessons and challenges, using context (current exercise, last error type, failed test names). Not a chat: no free-text input in the MVP.	P0
FR-G2	Authored content only: error-type explanations, per-exercise hints, per-test feedback messages, concept glossary. This is the whole MVP tutor.	P0
FR-G3	Provider-agnostic server endpoint; keys server-side only; per-user and global quotas; feature flag.	P1
FR-G4	LLM mode only: Socratic first, progressive hints, no full solutions for graded challenges unless the learner has completed them.	P1
FR-G5	Practice question generation.	P2

H. Auth and progress

ID	Requirement	Pri
FR-H1	Register (email and password only), age gate at signup, login, logout, password reset, email verification.	P0
FR-H2	Profile (display name, avatar choice, timezone).	P0
FR-H3	Progress, submissions, XP, badges, streak persisted and restored on any device.	P0
FR-H4	Guest mode: try the first lesson; progress migrates on signup.	P1
FR-H5	Data export and account deletion.	P0

I. Content management

ID	Requirement	Pri
FR-I1	Content in repo (`/content`), validated by schema in CI and a local `npm run content:check` command.	P0
FR-I2	Draft/published flag per item.	P0
FR-I3	Admin dashboard.	P2

7. Non-Functional Requirements

Category	Requirement
Performance	LCP ⩽ 2.5 s on landing (4G, mid-range device). Pyodide loads lazily, only on lesson/challenge pages, and is cached by the browser. Initial JS on landing ⩽ 200 KB gzipped (target).
Accessibility	WCAG 2.2 AA target: keyboard-operable everything (including editor escape/focus traps), visible focus, 4.5:1 text contrast, reduced-motion support, screen-reader labels for results.
Responsiveness	Breakpoints for mobile, tablet, desktop; no horizontal scroll.
Security	Server-authoritative rewards; row-level security on all user tables; secrets only in server env; rate limits on write and AI endpoints.
Reliability	Graceful degradation if the runtime fails to load (retry, clear message); progress writes are idempotent and retried.
Privacy	Collect minimum data; no third-party trackers without consent; minors considered (see TRD).
Maintainability	TypeScript strict mode, linted, tested; modular monolith; single source of truth for content, XP rules.
Browser support	Latest two versions of Chrome, Edge, Firefox, Safari; mobile Chrome and Safari.

Category	Requirement
Cost	MVP within free tiers with hard usage caps. Exact limits *verify* with official docs.
Observability	Structured server logs, client error reporting (free tier), run-failure metrics.

8. Key User Journeys and Acceptance Criteria

J1: First run (guest). Given a visitor on the landing page, when they click Start and press Run on the sample, then output appears in within the R1-D budgets (warm cache and cold load), with a visible loading state.

J2: Lesson to completion. A signed-in learner opens a lesson, edits starter code, runs it, sees output, passes the exercise tests, and the lesson shows Complete; XP increases by the configured amount **exactly once** even if completion is triggered twice or from two tabs.

J3: Challenge submission. The learner runs visible tests, submits, receives "X of Y checks passed", earns XP on first pass only; later passes award 0 XP and say so.

J4: Persistence. After logout and login on another device, completed lessons, XP, level, streak, and badges match.

J5: Failure handling. An infinite loop is stopped at the timeout, the UI says so, the editor stays usable, and the next run works without a page reload.

J6: Tutor. On a failing test, the learner opens the tutor and receives a hint tied to that test without the full solution.

J7: Tamper resistance. A forged request that awards XP directly, claims a completion without a valid start record, or replays a completed claim is rejected or awards 0 XP. A forged claim that passes every server check is an accepted, documented limitation of self-reported mode.

9. Gamification Rules (proposed, tunable in config)

Event	XP	Notes
Complete lesson (first time)	20	Once per lesson
Pass challenge, Easy / Medium / Hard (first time)	30 / 60 / 100	Hint penalty optional (pending decision)

Event	XP	Notes
Daily streak day	10	Once per local day, requires a qualifying activity
Finish module	50 bonus	Once

Levels: level N requires cumulative XP from a published curve (e.g., gentle early growth). **Badges** (bronze/silver/gold example): *Consistent* (3 / 7 / 30-day streak), *Problem Solver* (MVP thresholds 3 / 6 / 8 challenges, matching launch content), *Clean Coder* (first-try passes), *Explorer* (modules finished). All rules are shown to users on a Rules page.

Anti-abuse: XP derived from an append-only ledger with a unique key per (user, source, source_id); no client-supplied XP values; server verifies an attempt record exists.

10. UI/UX Requirements (summary; detailed page specs in Part 2)

- **Identity:** light theme, mint primary, coral accent, warm neutrals. Coral used for emphasis and rewards only; mint for primary actions and progress. No heavy gradients or glassmorphism; modest radius and low-elevation shadows.
- **Pages:** Landing, Sign up/Login, Dashboard, Course map, Lesson, Challenge list, Challenge, Achievements, Profile/Settings, Rules, 404/error.
- **Principles:** one primary action per screen; code and output get the most space; celebratory motion is short and respects reduced-motion; every list has an empty state; every async action has loading and error states.

11. Risks (product level)

Risk	Likelihood	Impact	Mitigation
Browser-run grading can be bypassed by determined users	High	Medium	Server-authoritative ledger, idempotency, plausibility checks; no public leaderboard until trusted grading exists; be transparent that it is not a secure sandbox
Pyodide download size hurts low-bandwidth users	High	Medium	Lazy load, service-worker caching, loading UX, "lite" lessons that need no runtime

Risk	Likelihood	Impact	Mitigation
Content creation is the real bottleneck	High	High	Schema, templates, authoring checklist; ship one excellent course
Free-tier limits or terms change	Medium	Medium	Replaceable integrations, usage caps, monitoring
Generic-looking UI	Medium	Medium	Design tokens first, design review gate per page
Minors' data	Medium	High	Data minimisation, no ads/trackers, review applicable law before launch

12. Decisions Requiring Clarification

1. Final product name and domain.
2. Hint cost (R1 default: free in the MVP; revisit with launch data).
3. Streak freeze policy (R1: none in the MVP).
4. Social login (R1: not in the MVP; which provider after launch?).
5. Is a public leaderboard required at launch? (Recommended: no.)
6. Age policy: minimum age, parental consent approach.
7. Who writes the first course, and what is the target length?
8. Preferred AI provider for v1.2 and monthly budget cap.
9. Hosting region preference and data-residency expectations.
10. Analytics: acceptable tools and consent approach.

Part 2: Technical Requirements Document (TRD)

1. Architecture Overview

A **modular monolith** on Next.js (App Router, TypeScript). The browser does the heavy lifting for code execution; the server owns identity, progress, rewards, and the AI tutor. Supabase provides Postgres, Auth, and row-level security (RLS). There are no microservices and no queue in v1.

```
flowchart LR
  subgraph Browser
    UI[Next.js UI + Monaco]
    W[Web Worker: Pyodide]
    UI <-->|postMessage| W
  end
  subgraph Vercel[Next.js on Vercel]
    RSC[Server Components]
    API[Route Handlers /api/*]
    CT[(Content: JSON/MDX in repo)]
    RSC --> CT
  end
  subgraph Supabase
    AUTH[Auth]
    DB[(Postgres + RLS)]
  end
  LLM[(Configurable AI provider)]
  UI --> RSC
  UI --> API
  API --> AUTH
  API --> DB
  RSC --> DB
  API -.optional.-> LLM
```

Key principle: public lesson content ships with the build; everything in the bundle, including all tests, is readable by the learner and is not secret. Reference solutions are kept out of the client bundle and released through an API only after completion or repeated attempts, as a pacing aid rather than a security control (if the repository is public, treat solutions as public).

2. Technology Decisions

Technology	Purpose	Verdict
Next.js + React + TypeScript	Routing, SSR/SSG, API routes in one codebase	**Keep.**
Tailwind CSS	Utility styling with design tokens	**Keep.**
shadcn/ui (Radix)	Accessible primitives copied into the repo	**Keep.** Only add components as needed.

Technology	Purpose	Verdict
Motion	Short reward/transition animations	**Keep, optional.** Lazy-load; CSS transitions suffice for most.
Monaco Editor	Code editor	**Keep**, lazy-loaded; weigh a lighter alternative (CodeMirror 6) if mobile performance suffers.
Pyodide + Web Worker	In-browser Python	**Keep.** Pin the version; self-host or use a CDN with caching (decide in Phase 4).
Supabase (Postgres + Auth)	Persistence and authentication	**Keep.** Free-tier limits and project pausing behavior: *verify*.
Zod	Runtime validation of content, API inputs	**Add.** Required for schema validation.
Vitest + Testing Library + Playwright	Unit, component, E2E	**Keep.** Add `axe` for accessibility checks.
Vercel	Hosting	**Keep.** Free-tier terms (including use limits) *verify*.
Rate limiting store	Abuse protection	Start with Postgres-based counters; consider a managed Redis only if needed.
Error monitoring	Client/server errors	Optional free tier (e.g., Sentry); *verify*.

Avoid in v1: separate backend service, GraphQL, state-management libraries beyond React state plus a small store if needed, ORM heavier than Supabase client plus generated types, microservices, server-side code execution.

Migration notes: Postgres is portable; Auth migration is the costly part (password hashes, sessions), so keep user-profile data in our own `profiles` table keyed by user id.

3. Repository Structure

```
/
├─ AGENTS.md
├─ content/
|  └─ python-basics/
```

```
|     ├─ course.json
|     └─ 01-first-steps/
|        ├─ module.json
|        ├─ 01-hello-world.lesson.json
|        └─ challenges/two-sum-lite.challenge.json
├─ src/
|  ├─ app/
|  |  ├─ (marketing)/page.tsx            # landing
|  |  ├─ (auth)/login, signup, reset
|  |  ├─ (app)/dashboard, courses/[course]/[module]/[lesson]
|  |  ├─ (app)/challenges, challenges/[slug], achievements, profile
|  |  └─ api/ progress, submissions, tutor, hints, health
|  ├─ components/ ui/ (shadcn), editor/, lesson/, challenge/, gamification/,
layout/
|  ├─ features/ lessons/, challenges/, progress/, tutor/   # domain logic, no
UI
|  ├─ lib/ supabase/, content/, execution/, xp/, rateLimit/, env.ts
|  ├─ workers/ pyodide.worker.ts
|  └─ styles/ tokens.css
├─ supabase/ migrations/, seed.sql, tests/
├─ scripts/ validate-content.ts
├─ tests/ e2e/, fixtures/
└─ docs/ (PRD, TRD, ADRs)
```

4. Content Model and Schemas

Content is JSON (or MDX body referenced from JSON), validated by Zod at build time and in CI. **Everything bundled with the app is public to the learner, including all tests.** Reference solutions live in separate files that are excluded from the client bundle and returned by an API only after completion or repeated attempts; this is pacing, not secrecy.

```
// Public content (bundled)
interface Lesson {
  id: string;              // stable, never reused: "py-basics.first-
steps.hello-world"
  slug: string;
  title: string;
  status: "draft" | "published";
  objectives: string[];
  estimatedMinutes: number;
  body: Block[];           // markdown | codeExample | callout | quiz
  exercise?: {
    instructions: string;
    starterCode: string;
    visibleTests: TestCase[];
```

```
    hints: string[];       // ordered, progressive
    xp: number;
  };
  next?: string; prev?: string;
}

interface Challenge {
  id: string; slug: string; title: string;
  difficulty: "easy" | "medium" | "hard";
  statement: string;       // markdown
  examples: { input: string; output: string; explanation?: string }[];
  starterCode: string;
  visibleTests: TestCase[];
  hints: string[];
  xp: number;
  tags: string[];
  additionalChecks: TestCase[]; // bundled with content: NOT secret (practice
check only)
}

type TestCase =
  | { kind: "stdout"; stdin?: string; expected: string; compare?: "exact" |
"trimmed" }
  | { kind: "call"; fn: string; args: unknown[]; expected: unknown; compare?:
"equal" | "approx" };
```

Validation rules (enforced by `npm run content:check`): unique and stable ids, resolvable prev/next links, XP within allowed ranges, at least one visible test per exercise, hint order defined, no draft content in production builds, reference solution must pass all of its own tests (run through Pyodide in CI).

5. Code Execution Design

5.1 Lifecycle

```
sequenceDiagram
  participant U as Learner
  participant UI as Editor UI
  participant W as Pyodide Worker
  U->>UI: Click Run
  UI->>W: {id, code, stdin, tests, timeoutMs}
  Note over UI: start watchdog timer
  W->>W: load runtime (first time), reset namespace
  W->>W: capture stdout/stderr, exec code, run tests
  W-->>UI: {id, status, stdout, stderr, traceback, testResults, durationMs}
  alt watchdog fires
    UI->>W: terminate()
    UI->>UI: spawn fresh worker, show "Timed out"
  end
```

5.2 Requirements

- One worker per page; runtime initialised lazily and cached by the browser.
- Fresh global namespace per run; `sys.stdout/stderr` redirected and size-capped (e.g., 64 KB).
- **Timeout via watchdog on the main thread**: `worker.terminate()` is the only reliable stop for infinite loops. Default 5 s, configurable per exercise.
- Stop button uses the same terminate-and-respawn path.
- `input()` supported through pre-supplied stdin in v1 (interactive input deferred).
- Memory: the browser tab is the limit; Pyodide cannot enforce a precise memory cap. Mitigate with output caps, recursion limit, and restarting the worker after errors. Document this honestly.
- No network or file system access is promised; unsupported packages show a clear message.
- Run tests by injecting a harness that calls the learner's function and compares results, returning structured results rather than parsing text.

5.3 Honest limitations (shown in product copy)

Browser execution protects *our servers*, not the learner's own device from their own code, and it is **not** an isolated server sandbox. Because evaluation happens client-side, a technically skilled user can alter results. See section 6 and R1-A.

6. Challenge Evaluation and Reward Integrity

Problem: grading runs in the learner's browser, so the client can report any result, and any test sent to the browser can be read by its user. Encrypting tests in transit does not help, because the browser must decrypt them to run them. R1 therefore stops calling any browser-delivered test "hidden".

Decision (ADR-001, revised): three trust tiers.

1. **Tier A, practice check (MVP, untrusted).** All tests, including extra edge-case checks, ship with the content and run in the Web Worker. This is educational feedback; the UI says "practice check", never "verified".
2. **Tier B, self-reported completion (MVP, server-recorded).** After a passing practice check the client posts a completion claim. The server validates it (R1-A), records it once in the XP ledger with `trust_level = self_reported`, and applies XP and streak rules. It proves the request was well-formed and unique, not that the code passed.
3. **Tier C, trusted grading (post-MVP).** Learner code runs in an isolated judge outside the main app server, with tests that never reach the browser. Required before leaderboards, certificates, competitions, or any score shown to a third party. Provider and cost are unresolved (D2).

Limitations of the free MVP: a skilled learner can inflate their own XP or complete challenges without solving them; there are no secret tests; XP is motivation, not a credential. Mitigations: server-configured rewards, one reward per source, plausibility checks, rate limits, a public rules page that says progress is self-reported, and ledger trust levels so Tier C events can be separated later. Full flow, ledger and checks: Part 3, R1-A and R1-B.

7. XP, Levels, Streaks, Achievements

- **Ledger-based:** `xp_events` is append-only with a unique key `(user_id, source_type, source_id)`; total XP = `SUM(amount)`; level computed from total using the configured curve (pure function, unit-tested).
- **Streaks:** computed from `activity_days` (one row per user per local date, derived using the profile timezone). Current streak = consecutive days ending today or yesterday. No freezes in the MVP (R1-C); a later freeze feature would need a `streak_freezes` record.
- **Achievements:** rule definitions in code/config (`criteria` evaluated server-side after relevant events); awards stored in `user_badges` with a unique `(user_id, badge_id, tier)` constraint.

- **Atomicity:** award logic executes in a single Postgres function (RPC) called by the server so XP insert, activity-day upsert, and badge checks succeed or fail together.
- **Timezone edge cases** (DST, travel) covered by tests.

8. Security and Privacy Plan

8.1 Threat model

Threat	Mitigation	Verification
Exposed API keys	Secrets only in server env vars; only the Supabase URL and anon key are `NEXT_PUBLIC_`; secret scanning in CI; `.env*` gitignored	CI secret scan; test that the service key is absent from the client bundle
Unauthorized DB access	RLS on every table, default deny; service-role key only in server code; award functions live in a schema not exposed through the Data API and have EXECUTE revoked from `anon` and `authenticated`	SQL tests as anon, user A, user B (R1-E)
Broken authN/authZ	Supabase Auth sessions via secure cookies; every route handler verifies the user; `user_id` always comes from the verified session, never the request body	API tests with no token, wrong user, expired session
Student code risks	Runs in a Web Worker in the learner's own browser, never on our server; honest limitation copy	Execution tests (infinite loop, huge output, import abuse)

Threat	Mitigation	Verification
Forged or inflated completion (XP manipulation)	Tier B only: server-config rewards, one reward per (user, source), start record required, plausibility checks, rate limits, `trust_level` on every ledger row. Residual risk accepted and documented (R1-A)	Replay, forgery and parallel-request tests
Duplicate submissions	Idempotency key plus a unique constraint in the ledger	20 parallel claims award once
Test and solution exposure	No secret tests exist in the MVP; solutions are released by API only after completion or repeated attempts; assume a public repo means public content	Authorization tests on the solution endpoint
AI endpoint abuse	Not applicable at launch (no LLM, no free-text endpoint); any later LLM endpoint needs quotas first	Gate in the post-launch checklist
Rate limiting and resource exhaustion	Per-route limits (R1-E), body size caps, code length cap (20 KB), worker timeouts	Rate-limit tests; load smoke test
Input validation and output handling	Zod on every input; learner output rendered as text; sanitized Markdown with no raw HTML; CSP header	Unit tests; CSP report review
Dependency vulnerabilities	Lockfile, audit job in CI, minimal dependency policy	CI audit
CSRF	SameSite cookies and origin checks on mutating routes	API tests
Backup and recovery	Supabase's official pricing page lists automatic backups as not included on Free, so the MVP runs its own scheduled export and a restore drill (R1-G)	Restore drill before launch

Threat	Mitigation	Verification
Children's data	Age gate, data minimisation, no trackers, pseudonymous display names, consent decision D1 (R1-F)	Privacy checklist, counsel review

8.2 Privacy

- Collect only email, generated or filtered display name, timezone, age band, and learning data. No ads, no third-party trackers, no behavioural analytics on learners.
- School-age learners are in scope (A6). The launch path for under-18 users is an open legal decision (D1); R1-F gives the options and the recommended default.
- Provide data export and account deletion at launch, with documented retention.
- Nothing is sent to an AI provider at launch. Any later LLM tutor needs a separate privacy review, and must not receive email or profile data.

9. Database Schema (Postgres)

```
erDiagram
  profiles ||--o{ attempts : starts
  profiles ||--o{ lesson_progress : has
  profiles ||--o{ submissions : makes
  profiles ||--o{ xp_events : earns
  profiles ||--o{ user_badges : holds
  profiles ||--o{ activity_days : logs
  profiles ||--o{ consent_records : accepts
  attempts ||--o| xp_events : "may produce"
```

Lessons, challenges, badge definitions and reward amounts live in versioned content and config in the repository, not in database tables. The server validates ids against the bundled content map. Database tables hold only per-user state.

Table	Key columns	Notes
`profiles`	`id` (= auth.users.id), `display_name`, `avatar`, `timezone`, `age_band`, `created_at`	User may update only `display_name`, `avatar`, `timezone` (column-level grants). `age_band` is set at signup by the server and is not user-editable.

Table	Key columns	Notes
`consent_records`	`user_id`, `policy_version`, `accepted_at`, `guardian_status` (nullable, per D1)	Insert by server only.
`attempts`	`id`, `user_id`, `source_type`, `source_id`, `content_version`, `started_at`, `claimed_at`	A start record created when an exercise is opened. One claim per attempt.
`submissions`	`id`, `user_id`, `challenge_id`, `attempt_id`, `code`, `code_hash`, `checks_passed`, `checks_total`, `duration_ms`, `created_at`	Keep the latest 3 per user and challenge; code capped at 20 KB.
`lesson_progress`	`user_id`, `lesson_id`, `status`, `completed_at`, `content_version`, unique `(user_id, lesson_id)`	Written by server only.
`xp_events`	`id`, `user_id`, `source_type`, `source_id`, `amount`, `trust_level`, `rule_version`, `created_at`, unique `(user_id, source_type, source_id)`	Append-only ledger. `trust_level` is `self_reported` or `trusted`.
`activity_days`	`user_id`, `local_date`, unique pair	Basis for streaks.
`user_badges`	`user_id`, `badge_key`, `tier`, `awarded_at`, unique `(user_id, badge_key, tier)`	Server-written.
`security_events`	`user_id`, `type`, `detail`, `created_at`	Rejected or suspicious claims; 30-day retention (assumption).
`rate_limits`	`key`, `window_start`, `count`	Atomic upsert counters.

Storage estimate (assumption): 1,000 learners with 8 challenges and 3 retained submissions of about 2 KB each is roughly 50 MB, well inside the 500 MB Free database allowance listed on Supabase's pricing page, but the ledger, activity and security tables grow with use and need a size alert.

RLS and permission rules: RLS is enabled on every table with no policies by default. `authenticated` may `SELECT` only its own rows in `profiles`, `attempts`, `submissions`, `lesson_progress`, `xp_events`, `activity_days`, `user_badges`. `authenticated` has no `INSERT`, `UPDATE` or `DELETE` on any of those except the three profile columns above.

`security_events`, `rate_limits` and `consent_records` have no client access. All reward writes go through one server-only function that takes `user_id` from the verified session; it lives outside the exposed schema and has EXECUTE granted only to the server role, because anything callable through the Data API can be called by any signed-in user with arbitrary arguments. A leaderboard, when it ships, uses a view exposing display name and trusted XP only.

Level and total XP are derived, not stored, to avoid drift (a materialised summary can be added if queries become slow).

10. API Specification

All routes are JSON, validated with Zod, require an authenticated session unless noted, and return a consistent error shape:

```
{ "error": { "code": "RATE_LIMITED", "message": "Try again in 30 seconds" } }
```

Method and path	Purpose	Request	Response
`GET /api/me`	Profile, XP, level, streak, badges	none	`{profile, xp, level, streak, badges[]}`
`PATCH /api/me`	Update display name, avatar, timezone	`{displayName?, avatar?, timezone?}`	`{profile}`
`GET /api/progress`	Completion map	none	`{lessons:{id:status}, challenges:{id:status}}`
`POST /api/exercises/:type/:id/start`	Create a start record (`type` is `lesson` or `challenge`)	none	`{attemptId, contentVersion}`
`POST /api/exercises/:type/:id/claim`	Claim completion after a passing practice check	`{attemptId, checks:{passed,total}, durationMs, code?, codeHash}`	`{status:"awarded"|"already_awarded"|"rejected", xpAwarded, level, streak, newBadges[]}`
`GET /api/challenges/:slug/solution`	Reference solution, released only if completed or the release rule is met	none	`{solution}` or 403
`GET /api/challenges/:slug/submissions`	Own history	none	`{submissions[]}`
`POST /api/account/export`	Data export	none	JSON file

Method and path	Purpose	Request	Response
`DELETE /api/account`	Delete account and data	`{confirm:true}`	`{deleted:true}`
`GET /api/health`	Public health check	none	`{ok:true}`

The Help panel needs no endpoint: hints, error explanations and per-test feedback ship with the content. The LLM tutor endpoint is post-launch.

Rules: all inputs validated with Zod; `attemptId` doubles as the idempotency key, and the ledger's unique constraint is the real guarantee; `xpAwarded` is computed only from server config; a repeated claim returns 200 with `already_awarded` and 0 XP; 429 with `Retry-After` on limits; request bodies capped (code at most 20 KB); the claim endpoint never trusts `user_id`, `xp` or `level` from the client. Rate limits per route: R1-E.

```
sequenceDiagram
  participant C as Client
  participant S as Next.js API
  participant D as Postgres
  C->>S: POST /exercises/challenge/:id/start
  S->>D: insert attempt (user, source, content_version)
  S-->>C: attemptId
  C->>C: run practice check in worker (Tier A, untrusted)
  C->>S: POST /exercises/challenge/:id/claim {attemptId, checks, durationMs,
codeHash}
  S->>S: auth, rate limit, schema, content version, plausibility
  S->>D: record_completion(user from session, attemptId)
  D-->>S: awarded or already_awarded (unique ledger key)
  S-->>C: feedback, XP, streak, badges (Tier B, self_reported)
```

11. AI Tutor Design

MVP: the Help panel (launch-mandatory). It makes no server calls, uses no LLM, and has no free-text input. It has four parts, all authored by the content writer and shipped with the content:

1. **Hints:** one to three ordered levels per exercise, revealed one at a time. Hints are free.
2. **Python error explanations:** a client-side mapper from exception type, and a few message patterns, to a plain-language explanation and a "what to check" prompt, stored in `content/errors.json`. Covers the common beginner errors plus our own timeout and output-limit messages.

3. **Exercise-specific feedback:** each test case may carry a `feedbackOnFail` message tied to that check.
4. **Concept glossary** linked from lessons.

The panel never shows a full solution; the solution is a separate gated action (`GET /api/challenges/:slug/solution`).

Post-launch, optional: LLM tutor. Sits behind a `TutorProvider` interface and a feature flag that is off by default. Before it ships it needs per-user and global quotas, token and timeout caps, a kill switch, prompt-injection handling (learner code and messages are data), no personal data in prompts, a privacy review for under-18 users, and automatic fallback to the Help panel. It is deliberately not designed further now.

12. UI/UX and Design System Specification

Tokens (initial, to be tuned in Phase 1):

Token	Value / rule
Primary (mint)	Teal-leaning mint such as `#14B8A6` family; text on mint must pass 4.5:1 (use dark ink, not white, if needed)
Accent (coral)	`#FF6B5E` family; rewards, streak flame, destructive-adjacent emphasis only
Neutrals	Warm gray scale 50 to 900; background near-white, not pure white everywhere
Semantic	Success, warning, error, info with icon plus text (never color alone)
Type	One sans family for UI, one monospace for code; scale 12/14/16/18/24/32/44; line height 1.5 body
Spacing	4 px base scale (4, 8, 12, 16, 24, 32, 48, 64)
Radius / shadow	6 px controls, 10 px cards; two elevation levels only
Motion	120 to 250 ms; XP count-up and badge unlock only; honors `prefers-reduced-motion`

Core components: Button (primary, secondary, ghost, danger; hover, focus, active, loading, disabled), Input, Card, Badge, ProgressBar, Tabs, Dialog, Toast, Tooltip, EmptyState, Skeleton, CodeEditor, OutputPanel, TestResultList, XPBar, StreakChip, BadgeTile, TutorPanel.

Page specifications

Page	Layout and behavior
Landing	Hero (headline, subcopy, Start CTA, live mini-editor), course preview cards, three feature highlights, how-it-works strip, footer. Mini editor loads Pyodide only after interaction.
Sign up / Login	Single column form, inline validation, password reset link, clear error states; no dark patterns.
Dashboard	Top: greeting, XP bar with level, streak chip. Main: Continue card (largest), course progress, recent achievements, recommended next. Secondary column collapses below main on mobile. Empty state for new users points to lesson 1.
Course map	Modules as vertical path; lesson nodes show done / current / locked; locked nodes explain how to unlock.
Lesson	Two panes on desktop: instructions left (objectives, steps, hints), editor + output right. Tabs on tablet; stacked with sticky Run bar on mobile. Footer: Prev / Next / Complete.
Challenge list	Filter by difficulty and status; cards show difficulty, XP, completion.
Challenge	Statement and examples left; editor, test results, output right; Run tests / Submit; hints drawer; tutor panel; result dialog with XP animation.
Achievements	Badge grid with tiers, locked badges show criteria.
Profile / Settings	Display name, avatar, timezone, export data, delete account.
Rules	Plain-language XP, level, streak, badge rules.
Error / 404 / Offline	Helpful message with next action.

Accessibility specifics: editor has documented keyboard escape from focus trap; test results announced via `aria-live`; focus management on dialogs; no information conveyed by color alone; touch targets at least 24 by 24 CSS px (WCAG 2.2 AA minimum), 44 px preferred.

13. Testing Strategy and Acceptance Criteria

Layer	Tools	What to cover
Unit	Vitest	XP/level curve, streak calculation (timezones, DST), content schema, test-case comparators, rule tutor matcher
Component	Vitest + Testing Library	Editor panel states, test result list, XP bar, dialogs, forms
Python execution	Vitest/Playwright (real Pyodide)	Hello world, syntax error, runtime error, infinite loop timeout, cancel, huge output cap, worker restart, `input()` stdin
Challenge evaluation	Vitest + Pyodide	Reference solution passes; known wrong solutions fail; edge cases
API	Vitest with test DB	Validation, auth required, rate limits, idempotency, error shapes
DB authorization	SQL tests (pgTAP)	Each table, as anon, user A, user B: allowed and denied matrix; `challenge_secrets` unreadable
Concurrency	Script	20 parallel submits award XP once
E2E	Playwright	Journeys J1 to J7 on Chromium, Firefox, WebKit; mobile viewport
Accessibility	axe + manual keyboard/screen-reader pass	Key pages, zero critical violations
Performance	Lighthouse CI	LCP, JS size, Pyodide lazy loading
Failure and recovery	Playwright	Runtime fetch failure, offline submit retry, expired token

Measurable acceptance for the core journey: open a lesson → edit code → run (output visible within the R1-D warm-cache budget) → submit → validation feedback within 2 s → XP awarded exactly once (verified by `xp_events` row count = 1 after duplicate and parallel submits) → sign out and in on a second browser → same XP, level, and completion.

CI gates: typecheck, lint, content check, unit and component tests, DB authorization tests, build, and E2E smoke on every PR. The agent must never report a check as passing unless it actually ran it.

14. Cost Scenarios

Facts below were checked against official pages on Oct 26, 2026 (sources in R1-G). Anything marked **verify** was not confirmed on an official page. No plan is assumed to stay free, and quotas can change.

	1. ₹0 MVP	2. Low-cost production	3. Growth-ready
Hosting	Vercel Hobby, which Vercel's docs restrict to non-commercial personal use; exceeding limits can block a feature for 30 days	Vercel Pro, required for any commercial use (price: **verify** on vercel.com/pricing)	Re-evaluate hosting and CDN
Database and Auth	Supabase Free: 500 MB database, 50,000 monthly active users, 2 active projects, pauses after 1 week of inactivity, automatic backups not included	Supabase Pro, listed from $25/month per organization with 8 GB database, 100,000 MAU, no inactivity pause; backup details **verify**	Larger compute, connection pooling, read replicas (**verify**)
Python runtime delivery	Pinned Pyodide from a public CDN or self-hosted; transfer is a cost on whichever side serves it (**verify** CDN terms)	Same	Own CDN if needed
Code execution	Browser only	Browser only	Isolated judge service for Tier C features (provider and cost unresolved, D2)
AI tutor	Authored Help panel only	Optional LLM with hard quotas (provider pricing **verify**)	Tiered quotas, cheaper model routing
Monitoring and backups	Free error-tracking tier (**verify**); own scheduled export	Paid tier if volume needs it	Full observability

	1. ₹0 MVP	2. Low-cost production	3. Growth-ready
Likely cost drivers	Developer time, domain	Hosting seat, database tier, domain	Bandwidth, database size, judge compute, AI tokens
Defer	LLM tutor, leaderboards, admin UI, timed rounds, trusted grading	Leaderboards until Tier C exists	Multi-language, classrooms

Cost controls: provider spend caps, in-app daily quotas, feature flags, alerts at usage thresholds, adapters so providers can be swapped, static content to keep database reads low.

15. Deployment and Environments

- **Environments:** local (Supabase CLI), preview (per PR), production. Separate Supabase projects for preview/staging and production.
- **Env vars (server):** `SUPABASE_SERVICE_ROLE_KEY`, `TUTOR_PROVIDER`, `TUTOR_MODEL`, `TUTOR_API_KEY`, `TUTOR_ENABLED`, `RATE_LIMIT_*`. **Public:** `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`. Validated at startup by `lib/env.ts`.
- **Migrations:** SQL files in `supabase/migrations`, applied by CI to staging first; destructive migrations need manual approval.
- **Security headers:** CSP (allowing Pyodide source and Monaco workers), HSTS, `X-Content-Type-Options`, `Referrer-Policy`.
- **Release:** merge to main → preview checks → manual promote to production; rollback by redeploying the previous build; content-only changes ship through the same pipeline.
- **Pre-launch checklist:** restore drill, RLS matrix green, secret scan clean, Lighthouse budgets met, legal pages (privacy, terms) reviewed.

16. Phased Roadmap

Effort is relative (S, M, L, XL), not a delivery promise.

Phase	Objective	Key outputs	Depends on	Acceptance / Definition of done	Effort
0	Inspect repo, confirm assumptions, start legal decision D1	Repo audit note, answered clarification list, D1 recorded	none	Decisions logged in `docs/decisions.md`	S
1	Spec, architecture, design system	PRD/TRD finalised, tokens, component sketches, AGENTS.md	0	Reviewer sign-off	M
2	Frontend foundation, landing	Next.js scaffold, Tailwind tokens, layout, static landing page	1	Lighthouse a11y ⩾ 95, responsive checks pass	M
3	Content and lessons	Zod schemas, content loader, lesson page, 3 sample lessons, `content:check`	2	Invalid content fails CI; lessons render from JSON	M
4	Pyodide spike, editor and execution	Loading spike and baseline (R1-D), worker with timeout/cancel, editor, output and test panels	3	Execution tests green incl. infinite loop; baseline recorded	L
5	Challenges and practice-check evaluation	Challenge page, comparators, `feedbackOnFail`, reference-solution CI check	4	Reference solutions pass; wrong ones fail	M
6	Auth, privacy basics and persistence	Supabase Auth, age gate, consent record, schema, RLS and grants, export/delete	3 (4, 5 for claims), 0 (D1)	Authorization matrix green	L
7	Ledger and gamification	Start/claim APIs, award function, XP, streaks, badges, toasts, dashboard	6	Duplicate and parallel claims award once	L
8	Help panel	Hints, error mapper, per-test feedback, glossary	4, 5	Panel never reveals a solution; mapper tests green	M
9	Content authoring tooling	Authoring guide, templates, lint rules (no admin dashboard)	3	New lesson added with no UI change	S
10	Hardening	Accessibility, security review, performance budgets, E2E suite	2 to 9	All CI gates green; manual audit done	L

Phase	Objective	Key outputs	Depends on	Acceptance / Definition of done	Effort
11	Deployment	Environments, headers, monitoring, backups and restore drill, launch checklist	10	Restore drill and smoke test pass	M
12 (post-launch)	Optional features	Google login, daily challenge, Tier C trusted-grading spike, leaderboard, LLM tutor	11	Each gated by its own checklist (R1-A, R1-F)	per item

Fast path to MVP: phases 0 to 7 with the authored Help panel only (Phase 8), one course, then 10 and 11. Phase 9 dashboard and LLM tutor are post-launch.

```
flowchart LR
  P0-->P1-->P2-->P3-->P4-->P5
  P3-->P6
  P5-->P6-->P7
  P4-->P8
  P6-->P8
  P3-->P9
  P7-->P10
  P8-->P10
  P9-->P10
  P10-->P11
```

17. Prioritized Backlog (first 20 tasks)

Priority meaning (revised): **P0** = launch-mandatory, **P1** = launch-optional (ship only if P0 is done and green), **P2** = post-launch.

#	Task	Pri	Effort	Depends on	Expected output	Acceptance
1	Repo audit and decisions log	P0	S	none	`docs/decisions.md`	Open questions answered or defaulted
2	Write AGENTS.md	P0	S	1	Project instructions	Reviewed
3	Scaffold Next.js, TypeScript, Tailwind, lint and test tooling	P0	S	2	Running app, CI skeleton	`build`, `lint`, `test` run in CI

#	Task	Pri	Effort	Depends on	Expected output	Acceptance
4	Age, consent and privacy decision (D1) plus draft policy	P0	S	1	Decision record, policy draft	Counsel input recorded; launch path for under-18 users chosen
5	Design tokens and base components	P0	M	3	`tokens.css`, ui components	Contrast verified, states present
6	Content schemas and validator	P0	M	3	Zod schemas, `content:check`	Bad fixtures rejected
7	Pyodide loading spike and performance baseline	P0	M	3	Spike report, CDN vs self-host and editor decisions	Baseline numbers recorded for R1-D profiles
8	Pyodide worker with timeout and cancel	P0	L	7	Execution module	Infinite-loop test passes
9	Editor panel and output UI (Monaco on large screens, textarea fallback)	P0	M	5, 8	Editor component	State matrix visible; keyboard escape works
10	Test harness, comparators, `feedbackOnFail`	P0	M	8	Evaluation lib	Unit suite green
11	Lesson renderer and course map	P0	M	5, 6	Lesson and map pages	Renders sample course
12	Challenge pages and 8 sample challenges (assumption)	P0	M	9, 10, 11	Challenges	Reference solutions pass in CI
13	Help panel: hints, error mapper, glossary	P0	M	9, 10	Panel	Mapper tests green; no solution shown
14	Landing page (static)	P0	M	5	Responsive page	Lighthouse targets met
15	Supabase schema, RLS, grants, authorization tests	P0	L	3	Schema and policies	Matrix green; award function not callable by clients
16	Auth flows, age gate, consent record	P0	M	15, 4	Signup, login, reset	Auth tests green
17	Rate-limit module, start and claim APIs, award function	P0	L	12, 15	Endpoints	Idempotency, forgery and parallel tests green
18	XP, level, streak, badges	P0	M	17	Gamification module	Unit and concurrency tests green

#	Task	Pri	Effort	Depends on	Expected output	Acceptance
19	Dashboard	P0	M	18	Dashboard page	Matches page spec
20	Rules, Privacy, Terms pages; data export and delete	P0	M	16	Pages and endpoints	Export and delete tested end to end
21	Scheduled backup export and restore drill	P0	S	15	Export job, drill notes	Restore succeeds on a clean project
22	Accessibility, security and performance gates	P0	M	19, 20	Fix list closed, CI budgets	Zero critical issues; budgets met
23	Deploy, monitoring, launch checklist	P0	M	21, 22	Production environment	Checklist complete
24	Landing mini editor	P1	S	7, 14	Demo	Loads runtime only on interaction
25	Local-only mode for guests (P0 if D1 chooses it for minors)	P1	M	16	Progress repository with local backend	No server data collected in local mode
26	Daily challenge	P1	S	18	Daily item	Awards once per day
27	Module unlocks	P1	S	18	Locked states	Unlock rules tested
28	Google login	P2	S	16	Provider config	Auth tests extended
29	Trusted-grading spike (Tier C)	P2	L	23	Decision record, cost estimate	D2 resolved
30	Leaderboard	P2	M	29	View and page	Counts `trusted` XP only
31	LLM tutor	P2	L	23, D1	Provider adapter	R1-F gate passed
32	Admin dashboard	P2	L	23	Admin UI	Revisit after about 50 lessons

Recommended execution order: 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10 → 11 → 12 → 13 → 14 → 15 → 16 → 17 → 18 → 19 → 20 → 21 → 22 → 23, then P1 items 24 to 27 as time allows, then P2 items.

18. AGENTS.md (starter)

```
# AGENTS.md

## Workflow
- Before changing anything: inspect the repo, read docs/ and existing
conventions.
- For each task: state a short plan, make small reviewable changes, run
typecheck, lint, tests and build, then report files changed and open issues.
- Never claim a test, build, or browser check passed unless you ran it and
saw the result.
- Do not overwrite working features without tracing their dependencies.
- Commit in small steps with clear messages; keep work on a branch.
- Ask for approval before: deleting files or data, running destructive
migrations, adding paid services, adding dependencies not in the TRD.

## Code
- TypeScript strict; no `any` without a comment. Zod-validate all external
input.
- Domain logic in `src/features` and `src/lib`; components stay
presentational.
- Content is data in `/content`; never hardcode lessons in components.

## Security
- No secrets in code or commits. Only NEXT_PUBLIC_ vars may reach the client.
- XP, progress, badges are written only by server code or RPCs. Never trust
client-sent rewards.
- Never run learner code on the server. Never present client-reported results
as verified; ledger rows from browser-run tests carry trust_level
self_reported.
- Every new table needs RLS and authorization tests.

## Design
- Use tokens from tokens.css; no one-off colors or spacing.
- Every async UI has loading, empty, and error states; WCAG 2.2 AA; respect
reduced motion.
- Avoid heavy gradients, glassmorphism, and decorative clutter.

## Testing
- New logic ships with tests. Bug fixes ship with a regression test.
- Run the relevant suites before reporting done; include commands and
results.
```

19. Risk Register (technical)

ID	Risk	L / I	Mitigation	Owner
T1	Client-side grading forged	H / M	ADR-001, no leaderboard, honest copy, upgrade path	Eng
T2	Pyodide load time and size	H / M	Lazy load, caching, loading UX, lite lessons	Eng
T3	Infinite loops or memory blowups freeze tab	M / M	Worker terminate, output cap, restart	Eng
T4	RLS misconfiguration leaks data	M / H	Default deny, authorization test matrix in CI	Eng
T5	Free-tier limits/terms change	M / M	Adapters, caps, monitoring, budget alerts	PM
T6	Monaco heavy on mobile	M / M	Lazy load, measure, CodeMirror fallback decision at Phase 4	Eng
T7	LLM cost spike or prompt injection	M / M	Quotas, kill-switch, solution-withholding policy tests	Eng
T8	Timezone streak bugs	M / L	Dedicated test suite, store local date explicitly	Eng
T9	Content quality bottleneck	H / H	Authoring templates, one excellent course first	PM
T10	Agent changes break existing features	M / M	Small PRs, CI gates, review checklist	Eng

20. Open Technical Decisions

1. Pyodide delivery: CDN vs self-hosted (affects CSP, caching, offline).
2. Monaco vs CodeMirror 6 after a mobile performance check.
3. Resolved by R1: no tests are treated as secret in the MVP, so no test-encryption scheme is needed.
4. Rate-limit store: Postgres counters vs managed Redis.
5. MDX vs plain JSON-with-Markdown for lesson bodies.
6. Supabase plan and region; backup strategy.
7. Error monitoring vendor, if any.

8. Whether guest progress is stored in `localStorage` only until signup.

Part 3: Revision R1 (focused corrections)

Scope: this revision corrects inconsistencies in Parts 1 and 2. Product vision, technology stack, mint-and-coral direction, core MVP features and the phase structure are unchanged. Earlier sections have been edited in place where they contradicted the decisions below. Items marked **assumption** are planning defaults, not facts. Nothing here has been implemented or tested.

R1-0 Inconsistencies identified

#	Where	Problem	Resolved in
1	TRD section 6, PRD FR-E3	Tests were described as hidden yet delivered to the browser, and "encrypted to the session". The browser must decrypt them to run them, so they are readable by the learner.	R1-A
2	PRD goal G3, risks	Progress called "trustworthy" and "server-authoritative" while results come from the client.	R1-A, R1-B
3	TRD section 9, AGENTS.md	`challenge_secrets` table and the rule "never expose challenge_secrets" assumed secret tests.	R1-A, section 9
4	PRD FR-F3 vs TRD section 7	Streak freeze marked "decision pending" in the PRD but a `streak_freezes` table was implied in the TRD.	R1-C
5	PRD MVP table, FR-H1, open question 4	"One social login (optional)" in scope, absent from requirements, and still a question.	R1-C
6	PRD FR-G1, MVP table, TRD API and Phase 8	A P0 "contextual tutor" with free text and an LLM mode conflicted with a rule-based MVP.	R1-C

#	Where	Problem	Resolved in
7	PRD sections 2 and 6, TRD section 9	Leaderboard listed as optional, deferred, and given a database view, with no launch gate.	R1-C
8	PRD J1, success metrics	Performance targets had no device or network definition; a cold Pyodide load on a slow connection cannot meet "under 10 s".	R1-D
9	TRD sections 14 and 8	Free-tier terms and backups left as "verify"; backups were assumed to exist.	R1-G
10	PRD section 5, roadmap	Too many items for one developer (guest migration, mini editor, unlocks, tutor adapter) sat inside the MVP.	R1-I
11	PRD A6, FR-H5	Minors were assumed but there was no age handling, consent path or retention; account deletion was P1.	R1-F
12	PRD NFRs, TRD section 12	Accessibility lacked editor-specific, timing and authentication rules; no alternative to Monaco.	R1-H
13	TRD section 10	No per-route rate limits.	R1-E
14	TRD section 9	Reward function could be called by clients if exposed; stored code was unbounded against a 500 MB database; `age_band` did not exist.	R1-B, R1-E
15	PRD FR-E4 vs badges	Hint cost undecided while a "first-try" badge depended on it.	R1-C
16	PRD section 9	Badge thresholds (5 / 25 / 100 challenges) were unreachable with 8 launch challenges.	R1-I

R1-A Grading model and submission flow

Tier	Where tests run	Can the learner read the tests?	What the server records	Allowed uses	In MVP
A. Practice check	Browser Web Worker	Yes, they ship with the content	Nothing	Instant feedback, hints, pass/fail display	Yes
B. Self-reported claim	Browser; server validates the claim	Yes	Ledger row with `trust_level = self_reported`	Personal XP, level, streak, badges, progress	Yes
C. Trusted grading	Isolated judge outside the main app server; tests never reach the browser	No	Ledger row with `trust_level = trusted`	Leaderboards, certificates, competitions, any score shown to someone else	No (D2)

What the free MVP can and cannot claim:

- It can say results are checked in your browser and XP records your own progress.
- It cannot say a solution is verified, that tests are hidden, or that rankings are fair.
- A skilled user can read the tests, hard-code outputs, or forge a claim that passes server checks. This is accepted for private, self-paced progress.
- Reference solutions are released after completion or repeated attempts. That is pacing, not security, and a public repository makes them public.
- Anything that compares learners or is shown to a third party waits for Tier C.

Revised challenge-submission flow (Tiers A and B):

1. The learner opens an exercise; the client calls `start`; the server stores an attempt with the content version.
2. The learner edits and runs code. The worker runs all bundled checks and shows a practice-check result.
3. When every check passes, the client calls `claim` with the attempt id, check counts, duration and code hash (code too, for challenges).
4. The server runs the verification steps in R1-B and, if they pass, calls the award function once.
5. The response says awarded, already awarded, or rejected, with XP, streak and badges when awarded.

```
flowchart LR
  subgraph Browser[Browser - untrusted]
    ED[Editor] --> WK[Worker: practice check, Tier A]
  end
  subgraph App[Next.js API - trusted]
    V[Validate claim, Tier B]
  end
  subgraph DB[Postgres - trusted]
    L[(xp_events ledger)]
  end
  J[Isolated judge, Tier C, post-MVP]
  WK -- claim --> V --> L
  ED -. future .-> J -.-> L
```

R1-B Ledger, idempotency, authorization and reward verification

Ledger rules

- `xp_events` is append-only. Event sources and `source_id`: `lesson_complete` (lesson id), `challenge_pass` (challenge id), `module_complete` (module id), `streak_day` (local date).
- Amounts come from a versioned config file on the server; the ledger stores `rule_version`. Changing amounts never rewrites past rows.
- The unique key `(user_id, source_type, source_id)` is the idempotency guarantee. The insert uses conflict-do-nothing and reports whether a row was created, so parallel requests and two tabs cannot double-award.
- Corrections are compensating negative events inserted by an operator (reason `reversal`, audited). Rows are never updated or deleted by learners.
- Total XP is the sum of the ledger; level is derived from total XP. Neither is stored independently.
- `trust_level` is recorded on every row so Tier C features can count only trusted XP later.

Claim verification, in order (any failure returns `rejected` with a generic code and writes a `security_events` row):

1. Authenticated session; `user_id` taken from the session only.
2. Rate limit (R1-E).
3. Body passes the Zod schema and size caps.
4. The attempt exists, belongs to this user, matches the exercise, and is unclaimed and less than 24 hours old (assumption).

5. The content version is the current or the previous published version.
6. `checks.total` equals the number of checks in that content version, and `checks.passed` equals the total. A partial result is stored as a submission but earns no XP.
7. Time since the attempt started is at least the exercise's authored `minSeconds` (default 20 s, assumption), and `durationMs` is consistent with it.
8. For challenges, store the submission and prune to the latest 3.
9. Call the award function, which inserts the ledger row, the activity day, streak and badge effects in one transaction.

Steps 4 to 7 raise the effort of casual abuse. They do not stop a scripted forgery, which is why Tier B stays labeled self-reported.

Authorization matrix

Resource	Anonymous	Signed-in learner, own rows	Signed-in learner, others' rows	Server function
Bundled content and tests	Read	Read	n/a	n/a
`profiles`	None	Read; update `display_name`, `avatar`, `timezone` only	None	Full
`attempts`, `submissions`, `lesson_progress`, `xp_events`, `activity_days`, `user_badges`	None	Read only	None	Write
Solution endpoint	None	Only if completed or the release rule is met	n/a	Read
`security_events`, `rate_limits`, `consent_records`	None	None	None	Read/write
Award function	None	None	None	Execute

R1-C Contradictions resolved

Topic	Decision	Revisit when
Streak freezes	None in the MVP. A streak is consecutive local days with at least one qualifying activity; a missed day resets it. No `streak_freezes` table.	After about two months of retention data (assumption)
Social login	None in the MVP. Email and password only, with email verification. Reason: one-developer scope and fewer third parties touching school-age data.	After launch; Google is the first candidate
Tutor capabilities	MVP = Help panel only: ordered authored hints, Python error explanations, per-test feedback, glossary. Not in the MVP: free-text chat, LLM answers, generated practice questions, level-adaptive explanations beyond hint levels.	After launch, behind the R1-F gate
Leaderboard	Not launched. Prerequisites: Tier C live, opt-in, pseudonymous display names, trusted XP only, privacy review for minors.	After D2 is resolved
Hint cost	Hints are free and do not change XP.	After launch data
Admin dashboard	Not in the MVP; content stays in the repository with CI validation.	After about 50 lessons
Unlockable areas	Not in the MVP; lessons are recommended in order but not locked.	P1 task 27

R1-D Pyodide loading strategy and performance budgets

Loading strategy

1. Load the runtime only on pages that run code. The landing page, dashboard and course map never fetch it. The optional landing mini-editor loads it on first interaction.

2. Show the lesson text and an editable editor immediately. Start fetching the runtime in a Web Worker within about 2 seconds after the lesson text has painted (assumption), so reading time overlaps download time.
3. On a data-saver setting or a very slow connection (as reported by the Network Information API where the browser supports it), do not prefetch. Show a "Load Python (about N MB)" button instead, with N measured in the spike.
4. Pin one Pyodide version. Serve it from a public CDN or self-host it, decided after the spike (D4). Provide one environment variable for the base URL so the choice is reversible.
5. Rely on normal HTTP caching in the MVP; a service worker for offline use is deferred because it adds update and cache-invalidation risk.
6. Keep one worker per page. After a timeout or Stop, terminate and recreate it; the replacement reuses cached files but must re-initialise, so show "Restarting Python".
7. Standard library only. No `micropip`, NumPy or other packages in the MVP. A content check rejects imports outside the supported list. Pyodide's docs state that some modules (for example `tkinter`, `turtle`, `curses`) are removed and others are unvendored, so exercises must be tested in the real runtime.
8. If the runtime fails to load, show a retry and a plain explanation. Lessons stay readable; code exercises are marked unavailable and earn no XP.
9. Cross-origin isolation is not adopted. Pyodide's docs say streaming download needs it, but it complicates embeds and third-party scripts, and the MVP loads fine without it.

Indicative size only: the Pyodide 0.23 release notes reported about 5.3 MB (Brotli) for Python plus the standard library, with about 1 s initialisation on a fast connection with caching off. That is an older version; the spike must measure the pinned one.

Test profiles (our own definitions; DevTools preset values are approximate and should be confirmed in the spike)

Profile	Definition
P1 Warm	4x CPU throttle, Fast 4G preset, runtime already cached
P2 Cold-slow	4x CPU throttle, Slow 4G preset, empty cache
P3 Real device	One physical Android phone in the ₹10,000 to ₹15,000 class (assumption, D8), run by hand before each release

Initial budgets (assumptions until the spike records a baseline; after that, CI fails on a regression of more than 20% against the baseline)

Metric	Budget	Profile	How measured
Landing LCP	⩽ 2.5 s	P2	Lighthouse CI
Landing JavaScript	⩽ 200 KB gzipped	n/a	Build size check
Lesson text visible and editor typeable	⩽ 4 s	P2	Playwright performance marks
Runtime transfer, cold	⩽ 8 MB compressed	P2	Network bytes in Playwright
Runtime ready, cold	⩽ 45 s with a progress indicator	P2	`pyodide:ready` mark
Runtime ready, warm	⩽ 4 s at p75	P1	`pyodide:ready` mark, 20 runs
First "hello world" after ready	⩽ 1 s	P1	Mark from Run click to output
Worker restart after timeout, warm	⩽ 4 s	P1	Mark from timeout to ready
Editor stays responsive during a 5 s busy loop	Keystroke echo ⩽ 200 ms	P1	Playwright typing test
Memory after 20 runs	Heap growth ⩽ 50 MB over baseline (assumption)	P1	CDP heap sample

Arithmetic note: at about 1.6 Mbps (a Slow 4G figure, approximate), 8 MB takes roughly 40 seconds, which is why the cold budget is 45 s with prefetch and a progress indicator rather than a few seconds.

R1-E Rate limits and database permission tests

These are our design defaults (assumptions), tuned after beta. They are not vendor quotas. Supabase Auth applies its own limits as well; check its current defaults in its documentation before launch.

Route	Limit	Key
Signup	5 per hour	IP
Login failures	10 per 15 minutes	IP and email

Route	Limit	Key
Password reset email	3 per hour	Email; 10 per hour per IP
`start`	120 per hour	User
`claim`	30 per minute, 300 per day; 6 per minute per exercise	User
Solution	20 per hour	User
`PATCH /api/me`	10 per minute	User
Export, delete	3 per day each	User
All JSON bodies	32 KB; submitted code 20 KB	n/a

Behavior: respond 429 with `Retry-After`. Counters use an atomic upsert in Postgres. If the counter store fails, fail closed on auth and `claim`, fail open on reads. Store only hashed IPs, kept for the window length.

Database permission tests (run in CI against a local Supabase): for every table, as anonymous, learner A and learner B, assert the matrix in R1-B; assert the award function returns permission denied through the Data API for `authenticated`; assert a learner cannot update `age_band` or any non-whitelisted profile column; assert RLS is enabled on every table in the schema (a migration check that fails when a new table lacks it).

R1-F Privacy and school-age learners

What I found (as of Oct 26, 2026). Law-firm summaries of India's DPDP Act, 2023 and the DPDP Rules, 2025 say: anyone under 18 is a child; processing a child's data needs verifiable parental or guardian consent; tracking, behavioural monitoring and targeted advertising aimed at children are barred; some classes of fiduciary, such as educational institutions, get conditional exemptions for limited purposes. The Rules were notified in November 2025, and the substantive duties, including children's consent, are reported to take effect about 18 months later (around May 2027). I read these through secondary summaries and search excerpts, not the Gazette text, so confirm with qualified counsel and the official Rules before launch. If the product is offered outside India, other children's-privacy laws may also apply (assumption A5 limits scope to India).

Options for under-18 learners (decision D1)

Option	What it means	Cost and risk
1. Adults only	Accounts for 18+; minors can read lessons with no account and no saved data	Smallest build; excludes the school audience
2. Local-only mode for minors	No server-side personal data; progress, XP and streak stored on the device only	Moderate build (backlog 25); no cross-device progress; XP is local and unverified
3. Guardian-consent flow	Verifiable parental consent before any account	Legal, cost and build complexity; defer until counsel confirms the method
4. School or institution partner	Institution acts for its students under the Rules' conditions	Needs agreements; later

Recommended default (assumption): Option 2 plus accounts for 18+. Backlog task 25 becomes P0, and D1 is decided in Phase 0 to 1. Self-declared age is not verification; the privacy notice must say so, and counsel should advise whether more is needed.

Data-minimisation rules for all learners

- Collect email, a generated display name (for example adjective, animal, number), timezone, age band and learning data. No free-text profile fields, public profiles or messaging.
- No advertising, no third-party analytics SDKs, no session replay. Only essential session cookies.
- No behavioural profiling beyond what streaks and progress need; counsel to confirm that XP, streaks and the Help panel count as ordinary service functions.
- Nothing goes to an AI provider at launch; a later LLM tutor is 18+ until reviewed.

Retention (assumptions, D10)

Data	Retention
Account and learning data	Until the user deletes the account, which hard-deletes all rows; accounts inactive for 12 months get a notice, then deletion
`security_events`	30 days
Rate-limit counters	The window length

Data	Retention
Backup exports	30 days, encrypted; deleted accounts disappear from backups when they expire, and the policy says so

Also required at launch: plain-language privacy notice and terms readable by a young teenager; consent record with policy version; export and delete working end to end; a named grievance contact; a written breach runbook (summaries mention a 72-hour reporting duty; verify with counsel); a processor list (Supabase, Vercel, CDN) with regions and data-processing terms checked (D14).

R1-G Free-tier and pricing verification

Checked Oct 26, 2026 against excerpts of the official pages below. Treat all of it as a snapshot that must be re-opened within a week of launch.

Claim	Result	Source
Supabase Free limits	500 MB database per project, 50,000 monthly active users, 2 active projects, paused after 1 week of inactivity, automatic backups and point-in-time recovery not included	Supabase pricing, pricing text
Supabase Pro	Listed from $25/month; includes 8 GB database and 100,000 MAU; backup specifics not confirmed here (**verify**)	Supabase pricing text
Vercel Hobby	Free but restricted to non-commercial personal use; commercial use needs Pro or Enterprise; exceeding limits can mean waiting 30 days	Hobby plan, fair use, limits note
Vercel numeric quotas and Pro price	Third-party articles disagree (for example on function invocations), so none are used here (**verify** on vercel.com/docs/limits and vercel.com/pricing)	n/a

Claim	Result	Source
Pyodide module and delivery constraints	Removed and unvendored stdlib modules; streaming download needs a worker plus cross-origin isolation	Deploying, Wasm constraints
Pyodide size	About 5.3 MB (Brotli) for Python plus stdlib in 0.23; pinned version must be measured	0.23 release notes
DPDP timeline and children's consent	Rules notified Nov 2025; substantive duties phased to about May 2027; read through secondary summaries	AMSS summary, SANSA Legal, Rule 12 note

Consequences for the plan: (1) Supabase Free pauses idle projects, so a quiet period can make the first visitor wait; the launch checklist needs a keep-warm decision or a move to Pro before public launch. (2) No automatic backups on Free, so backlog task 21 is mandatory. (3) Hobby is only valid if the launch is truly non-commercial; any revenue, sponsorship or paid institutional use means Vercel Pro (D3). (4) Nothing in this plan promises a permanently free service.

R1-H Accessibility additions

- **Editor:** Monaco on large screens, with its screen-reader mode and a documented shortcut that releases the Tab key from the editor. Below the large breakpoint, and for anyone who chooses it, a plain monospace textarea with Tab handling off. Both pass the same keyboard test.
- **Status messages:** run state, test results and achievements use a polite live region. Pass or fail is shown with an icon and text, never color alone.
- **Toasts:** auto-dismissing messages must remain reviewable, so achievements also appear in a persistent recent-activity list.
- **Motion:** XP count-up and badge animations respect reduced-motion settings.
- **Layout:** works at 200% zoom and 320 px width without two-dimensional scrolling; a sticky Run bar must not cover the focused element; interactive targets are at least 24 by 24 CSS px.
- **Authentication:** no cognitive-test CAPTCHA or memory puzzles; if bot protection is added later it must meet the WCAG 2.2 accessible-authentication rule.
- **Time limits:** timed rounds stay deferred; when added they need a user-adjustable limit.
- **Syntax theme:** code colors meet 4.5:1 contrast.

- **Manual checks before launch:** keyboard-only pass through signup, a lesson and a challenge; one screen-reader pass (NVDA on Windows and VoiceOver on iOS or macOS) on the same paths.

R1-I Mandatory vs optional scope (one developer, one coding agent)

Tier	Contents
Launch-mandatory (P0)	Static landing page; email and password signup with verification, age gate and consent record; the D1 under-18 path; one course of about 12 lessons and 8 challenges (assumption); editor and in-browser runner with timeout and cancel; practice-check feedback; Help panel; start and claim APIs with the ledger; XP, levels, daily streak (no freezes); 6 badges (Consistent 3 / 7 / 30 days, Problem Solver 3 / 6 / 8 challenges); dashboard with continue card, XP, streak, progress, recent achievements; Rules, Privacy and Terms pages; data export and delete; rate limits; RLS and authorization tests; scheduled backup export and restore drill; accessibility baseline; CI gates; deployment
Launch-optional (P1)	Landing mini-editor; local-only mode (becomes P0 if D1 chooses it); daily challenge; module unlocks; recommended-next logic beyond "next lesson"
Post-launch (P2)	Google login; trusted grading (Tier C); leaderboard; LLM tutor; admin dashboard; timed rounds; debugging dungeons; streak freezes; more courses; service-worker caching; practice-question generation; Clean Coder and Explorer badges

Scope guard rules: no P1 or P2 work starts until every P0 CI gate is green. An item enters P0 only if another leaves it. The largest hidden cost is writing the course, so content work is scheduled alongside Phases 3 to 5, not after them. The PRD's section 5 table and the backlog (section 17) are updated to match.

R1-J Acceptance criteria and tests for every changed requirement

Change	Acceptance criterion	Tests
Practice check versus verified (FR-E2, FR-E3, ADR-001)	The UI says "practice check" for browser results and never uses "verified" or "hidden"; no content field named for hidden tests exists	Schema test rejects such fields; E2E asserts the label text; bundle search for secret-test fields
Ledger and idempotency (FR-F1, R1-B)	20 parallel claims for one exercise create exactly 1 ledger row; a request body containing `xp` or `level` is ignored; awarded XP equals the config amount	Concurrency script; API test with tampered body; unit test for amounts
Claim verification (R1-B steps 4 to 7)	Each of these returns `rejected` with 0 XP: no attempt, another user's attempt, reused attempt, attempt older than 24 h, unknown content version, wrong check total, claim faster than `minSeconds`; a partial pass stores a submission but gives 0 XP	Table-driven API tests
Authorization (R1-B matrix, R1-E)	Every cell of the matrix holds for anonymous, learner A and learner B; the award function is not callable by clients; `age_band` is not editable	SQL authorization tests in CI
Solution release	403 before completion or the release rule; 200 after; every release is logged	API tests
Streaks without freezes (FR-F3)	A missed local day resets the streak to 1 on the next activity; DST and timezone-change cases pass; no `streak_freezes` table exists	Unit tests; migration inspection

Change	Acceptance criterion	Tests
Email-only auth (FR-H1)	Signup and login show no social buttons; the age gate must be answered; a consent record is stored with the policy version	E2E; API test
Help panel (FR-G1, FR-G2)	Opening the panel makes no network request; hints never contain the reference solution (content lint fails if 3 or more consecutive solution lines appear in a hint, assumption); every challenge has at least 2 hint levels; the error mapper covers each listed exception type; a failed check with `feedbackOnFail` shows that message	E2E network assertion; content lint; table-driven unit tests
Hints free (FR-E4)	Using any number of hints leaves XP unchanged	Unit and API test
Leaderboard not launched	No route, navigation entry or database view exists; flag is off	Route test; migration inspection
Badge thresholds	Every badge threshold is at most the number of available items	Unit test against content counts
Pyodide loading (R1-D)	The runtime is never fetched on landing, dashboard or course map; with data-saver emulated, nothing is prefetched until the button is pressed; blocking the runtime URL shows retry and read-only mode, and a code exercise earns no XP	Playwright network tests with route blocking
Performance budgets (R1-D)	Every metric in the budget table meets its budget on its profile; after the baseline, no metric regresses more than 20%	Lighthouse CI; Playwright marks under P1 and P2; manual P3 run before release

Change	Acceptance criterion	Tests
Rate limits (R1-E)	The request after each limit returns 429 with `Retry-After`; a limiter store failure rejects `claim` and auth requests	API tests with a stubbed store
Privacy (R1-F)	Export contains every row for that user; delete leaves zero rows across all tables and invalidates sessions; pages load no third-party origin other than the approved list; policy pages exist and are versioned; D1 path tested once decided	E2E; SQL row-count test; Playwright origin assertion
Accessibility (R1-H)	Zero critical axe violations on landing, signup, lesson, challenge, dashboard; the journey J2 completes by keyboard alone; reduced motion removes count-up animation; layout holds at 320 px and 200% zoom; the textarea editor passes the same journey	axe in CI; keyboard E2E; manual screen-reader pass
Backups (R1-G)	A restore of the latest export into a clean project reproduces row counts for every table	Documented restore drill before launch
Pricing claims (R1-G)	Supabase and Vercel pages are re-opened within 7 days of launch and differences are logged	Launch checklist item
Scope guard (R1-I)	Pull requests for P1 or P2 tasks are blocked while a P0 gate is red	Pull-request template and review checklist

R1-K Unresolved decisions

Resolved by R1 and removed from the open list: hint cost (free), streak freeze (none), social login (none in MVP), leaderboard at launch (no), secret-test encryption (not needed).

ID	Decision	Needed by	Default if unanswered
D1	Under-18 launch path and consent method (R1-F); counsel review	Phase 1	Option 2 plus 18+ accounts
D2	Trusted-grading provider, cost and timing (Tier C)	Before any leaderboard or certificate	Not built
D3	Is the launch non-commercial, so Vercel Hobby is allowed?	Before deployment	Assume commercial risk; budget for Vercel Pro
D4	Pyodide delivery: CDN or self-hosted	After spike (task 7)	Public CDN, pinned version
D5	Editor breakpoint and Monaco versus lighter editor	After spike	Monaco at 1024 px and wider, textarea below
D6	Product name and domain	Phase 2	"PyQuest" placeholder
D7	Who writes the course, and whether 12 lessons and 8 challenges is right	Phase 3	Developer writes it; sizes as stated
D8	Reference mid-range device and final performance budgets	After baseline	Android phone in the ₹10,000 to ₹15,000 class
D9	Rate-limit numbers	After beta	Table in R1-E
D10	Retention periods	Before launch	Table in R1-F
D11	`minSeconds` per exercise	Phase 5	20 s
D12	Backup frequency and storage destination	Phase 11	Daily export to private storage
D13	Revisit conditions for streak freezes and Google login	After launch	Two months of data

ID	Decision	Needed by	Default if unanswered
D14	Data region and processor terms for Supabase, Vercel and the CDN	Before launch	Closest available region; review terms
D15	Analytics at launch	Before launch	None
D16	Error-monitoring vendor and its privacy settings	Phase 11	Server logs only; client reporting off

Assumption register (unchanged by this revision unless noted): A1 to A8 in Part 1 section 2. New in R1: 24-hour attempt validity, 20-second minimum time, hashed-IP rate limit keys, 12-month inactivity deletion, 50 MB heap-growth budget, and the 12-lesson, 8-challenge course size.